

Befund: Dämpfer des Wachpostens

Befund: Der Dämpfer des Wachpostens verschluckt gleichartige Befunde

Stichtag: 2026-08-19 · **Gefunden:** 19.08.2026, 23:23 Uhr (Claudia) · **Anlass:** erster echter Lauf des Log-Wachpostens um 23:20 Uhr · **Für:** Engywuck (Bewertung) und Mick (Umsetzung)

Änderungsverlauf

2026-08-20 00:34 — Adams Auflage aufgenommen: Eine Frage darf nur gestellt werden, wenn sie im Chat beantwortbar ist **und** die Antwort wirkt; sonst entfällt sie. Der zweistufige Vorschlag wird dadurch zur Rangfolge — Wirkung zuerst, Umformulierung nur als Übergang.

2026-08-20 00:32 — Zweiter Befund ergänzt: Die Schlusszeile „Engywuck wecken?“ ist eine Frage ohne Empfänger; Adams Daumen am 20.08. um 00:19 Uhr blieb wirkungslos. Nachgemessen im Reaktions- und Sendepfad.

2026-08-19 23:40 — Erstfassung.

Die Kurzfassung

Der Wachposten hat bei seinem ersten Lauf sieben neue Zeilen der Fehlerdatei gelesen, **eine** davon gezeigt und die übrigen **sechs** stillschweigend verworfen — den Lesestand aber trotzdem ans Dateiende gesetzt. Damit sind sie dauerhaft weg. Gezeigt wurde ausgerechnet die **älteste** Zeile vom 27. Juli; die sechs jüngeren, darunter die Spur zu **fünf nie zugestellten Nachrichten**, blieben unsichtbar.

Der Posten hat also genau das getan, wogegen er gebaut wurde: eine Stille erzeugt, die von außen wie Ruhe aussieht.

Ein zweiter, unabhängiger Befund kam am folgenden Morgen dazu: Die Schlusszeile „Engywuck wecken?“ ist eine Frage ohne Empfänger — eine Reaktion darauf kann den Posten nicht erreichen. Er steht weiter unten.

Was gemeldet wurde — und was tatsächlich vorlag

Die Meldung um 23:20 Uhr lautete: „Eine neue Zeile in der Fehlerdatei“, gefolgt von der Zeile `2026-07-27 11:58:03 | postfach parse | JSONDecodeError`.

Tatsächlich standen in `logs/bot-errors.log` sieben Zeilen, alle ungelesen:

#	Zeitpunkt	Inhalt
1	27.07. 11:58	<code>postfach parse \ JSONDecodeError</code>
2	10.08. 22:48	<code>voice get_file (1. Versuch) \ Timeout</code>
3-5	16.08. 01:47	<code>postfach send \ Timeout (dreimal)</code>
6	16.08. 01:47	<code>postfach send \ NetworkError: httpx.ReadError</code>
7	16.08. 01:47	<code>postfach send \ RuntimeError('This HTTPXRequest is not initialized!')</code>

Der Merktzettel (`~/ .claude/wachposten/stand.json`) steht seither auf 570 — der vollen Dateigröße. Die Zeilen zwei bis sieben gelten als gelesen, ohne je gezeigt worden zu sein.

Der Mechanismus

Drei Bausteine greifen ineinander, jeder für sich richtig gedacht:

1. `wachmuster.treffer()` (**Zeile 80-81**): Für die Fehlerdatei gilt „jede neue Zeile ist der Befund“. Trifft kein Muster, vergibt sie die Pauschal-Kennung `fehlerdatei`. **Alle sieben Zeilen tragen damit dieselbe Kennung.**
2. `wachposten._daempfen()` (**Zeile 138-142**): Läuft über die Befundliste und setzt beim ersten Durchlasser `bekannt[kennung] = jetzt`. Für jeden weiteren Befund derselben Kennung ist `jetzt - bekannt[kennung]` praktisch null, also kleiner als die Wiedervorlagefrist von einer Stunde — er fällt durch `continue`. **Sechs Befunde verschwinden in derselben Sekunde, in der der erste gemeldet wird.**
3. `wachposten.lauf()` (**Zeile 193, 203**): Der Lesestand wird unabhängig davon ans Dateiende gesetzt und geschrieben. Was der Dämpfer verworfen hat, wird nie wieder gelesen.

Der Dämpfer beantwortet die Frage „habe ich **diese Sache** schon gemeldet?“ — richtig und nötig, damit ein stehender Fehler nicht zwölfmal je Stunde meldet. Faktisch beantwortet er aber eine zweite Frage mit: „wie viele **verschiedene** Zeilen zeige ich in einer Meldung?“ Dafür ist `MAX_ZEILEN` da, und diese Grenze läuft dadurch leer: Sie greift erst nach dem Dämpfer, wo nur noch ein Befund je Kennung übrig ist. Auch der Hinweis „und X weitere“ kann so nie erscheinen — er zählt nur die durchgelassenen, nie die verworfenen.

Was dabei verloren ging

Die sechs verschluckten Zeilen sind keine Kosmetik. Sie führen zu Nachrichten, die Adam nie erreicht haben — nachgesehen in `~/postfach/failed/`:

- **Fünf Stundenblumen-Meldungen** vom 16.08., zwischen 01:22 und 01:27 gelegt, um 01:47 sämtlich gescheitert. Drei an Zeitüberschreitungen, eine an einem Lesefehler, die fünfte an `This HTTPXRequest is not initialized!` — das Muster deutet auf einen Bot, der in diesem Moment herunterfuhr oder neu

startete. Fünf Befunde der Stundenblume sind damit nie angekommen.

- **Eine Ansage von mir** vom 27.07., 11:57 Uhr — die Ankündigung zur Umstellung der Dateinamen auf das Datumspräfix, mitsamt Dauerschätzung. Sie kam nie an; Adam hat an diesem Vormittag also auf eine Ankündigung gewartet, die im Postfach zerbrochen lag.

Beides wäre sichtbar geworden, hätte der Posten alle sieben Zeilen gezeigt.

Nebenfund: Woran der Auftrag vom 27.07. zerbrach

Die Datei `~/postfach/failed/1785146271313508170.json` enthält an der Bruchstelle die Zeichenfolge `... oft die guelt", "ige), ...` — mitten im Wort „gültige“ steht ein schließendes Anführungszeichen, ein Komma und ein neues öffnendes. JSON liest das als Beginn eines neuen Schlüssels und verlangt einen Doppelpunkt; daher die Meldung `Expecting ':' delimiter`.

Der Text der Datei ist durchgehend in ASCII-Umschreibung verfasst („Datumspraefix“, „pruefe“, „aussen“). Das ist derselbe Umweg — Text erst umschreiben, dann zurückersetzen —, den Adam am 28.07. bei den Umlauten beanstandet hat. Hier hat er nicht nur die Schreibweise verdorben, sondern den Auftrag selbst zerbrochen. Die Lehre der Umlaut-Regel bestätigt sich damit an einem zweiten Schaden: **Der Umweg ist die Fehlerquelle, nicht die Rückersetzung.**

Zweiter Befund: Die Schlusszeile fragt, aber niemand kann antworten

Jede Meldung des Postens endet mit „Engywuck wecken?“. Adam hat darauf am 20.08. um 00:19 Uhr mit einem Daumen reagiert — ohne jede Wirkung. Nachgemessen im Code, nicht vermutet:

Der Bot wertet eine Reaktion nur dann als Antwort, wenn zur betreffenden Nachricht eine **offene Frage registriert** ist. Registriert wird ausschließlich im Sendepfad (`bot.py` 6235, 8750, 8802) — also bei Nachrichten aus einem Modelllauf. **Der Postfach-Versand registriert keine Frage.** Damit fällt die

Reaktion in den Zweig „keine offene Frage, dazu ein Quittungs-Zeichen“ (`bot.py` 4040–4042) und löst die **stille Quittung** aus: ein Häkchen an der Nachricht, kein Lauf, keine Handlung. Das ist dort richtig gebaut — es trifft hier nur den falschen Fall.

Hinzu kommt: Einen technischen Weckruf für die Kontrollsituation gibt es ohnehin nicht, und ihre Richtung nach außen läuft **mit Absicht** über Adam (Vier-Augen-Prinzip, so im Bauauftrag vom 18.08. festgehalten). Die Frage suggeriert also eine Handlungsmöglichkeit, die es nicht gibt.

Adams Auflage vom 20.08., 00:31 Uhr — verbindlich, keine Option

Adam hat dazu eine Regel gesetzt, die über diesen Fall hinausreicht und **für jede künftige Automatik gilt, die ihm eine Frage stellt:**

Wer A sagt, muss auch B sagen. Eine Frage darf nur gestellt werden, wenn Adam sie **im Telegram-Chat beantworten kann** und die Antwort **eine Wirkung hat**. Fehlt eine der beiden Hälften, wird die Frage **nicht gestellt** — dann sagt die Nachricht schlicht, was der Stand ist.

Sein Wortlaut sinngemäß: Wenn die Antwort von hier aus nichts auslösen kann, brauche er die Frage nicht gestellt zu bekommen, das sei Irrsinn. Gewünscht ist ausdrücklich eine Schaltfläche oder wenigstens die Möglichkeit, mit „ja“ zu antworten — und dann muss auch etwas geschehen. **Begründung, die den Rang erklärt:** Eine Frage ohne Wirkung ist schlimmer als gar keine, weil er sich darauf verlässt, entschieden zu haben.

Damit dreht sich die Reihenfolge des ursprünglichen Vorschlags um. Nicht mehr „ehrlichere Schlusszeile jetzt, Wirkung vielleicht später“, sondern:

1. **Die Wirkung ist das Ziel.** Der Baustein existiert bereits — das Auftragsbuch (`auftragsbuch.py`) ist ausdrücklich „der Weg, auf dem ein Auftrag ohne Adams Hände ankommt“, mit Einstufung und Übergabe an die Arbeitsliste. Es ist mit der Wachposten-Meldung nur nicht verbunden.

2. **Bis die Wirkung steht, verschwindet die Frage.** Die Schlusszeile sagt dann, was der Stand ist und wo der Befund liegt — sie lädt zu keiner Antwort ein, die nirgends ankommt.

Zwei Dinge sind ungeprüft und dürfen nicht behauptet

werden: ob der Postfach-Weg Schaltflächen tragen kann (`_postfach_send_one` ist bis zum Ende zu lesen), und ob eine Reaktion die Kontrollsitzung **starten** darf. Letzteres ist keine technische, sondern eine Grundsatzfrage — das Vier-Augen-Prinzip hängt daran.

Die offene Entscheidung, Adam und Engywuck vorgelegt:

Soll ein „Ja“ die Kontrollsitzung tatsächlich **starten**, oder den Auftrag **verbindlich registrieren**, sodass sie ihn beim nächsten Start als Erstes vorfindet — das Öffnen aber bei Adam bleibt?

Vorschlag

1. Die Sperre gegen den Stand vor dem Lauf prüfen, nicht gegen den laufenden.

In `_daempfen()` die neuen Zeitstempel in ein getrenntes Verzeichnis schreiben und erst nach der Schleife in `bekannt` übernehmen. Dann entscheidet die Sperrfrist über Läufe hinweg, wie gewollt — aber innerhalb eines Laufs überleben verschiedene Zeilen derselben Kennung. Der Eingriff ist klein und berührt die Zeitlogik nicht.

2. Das Verworfenen zählen und nennen. `rest` darf nicht nur die Differenz zu `MAX_ZEILEN` sein, sondern muss die vom Dämpfer zurückgehaltenen Befunde mitzählen — mit eigener Formulierung, etwa „und N weitere, die der Dämpfer zurückhält“. Begründung: Was der Dämpfer schluckt, ist wegen des wandernden Lesestands **endgültig** weg. Die Zählzeile ist die einzige Spur, die davon bleibt.

3. Die Kennung für die Fehlerdatei feiner schneiden — zu prüfen, nicht gesetzt.

Denkbar wäre, den Teil vor dem ersten senkrechten Strich (`postfach parse`, `postfach send`, `voice get_file`) an die Kennung anzuhängen. Dann dämpft eine stehende Zeitüberschreitung weiterhin sich selbst, ein neuer Fehlertyp kommt aber durch. **Vorsicht:** Das erhöht die Meldemenge und kann die Postfach-Obergrenze von sechs

Nachrichten je Stunde reizen. Punkt eins und zwei lösen den akuten Schaden auch ohne diesen Schritt; er gehört bewertet, nicht nebenbei mitgebaut.

4. Die sechs verlorenen Zeilen einmalig nachholen. Der Lesestand steht auf 570 und lässt sich auf null setzen, damit die Datei einmal vollständig gelesen wird — sinnvollerweise erst nach Punkt eins, sonst wiederholt sich derselbe Vorgang.

Was kann brechen und wer merkt es

- **Der bestehende Prüfer bleibt grün.**

`_daempfer_meldet_nicht_zwoelfmal_je_stunde` in `scripts/test_wachposten.py` (Zeile 113) prüft **dieselbe Zeile in zwei aufeinanderfolgenden Läufen**. Da zwischen den Läufen der Stand geschrieben wird, greift die Sperre unverändert. Der Vorschlag ändert nur das Verhalten innerhalb eines Laufs — genau die Lücke, die dieser Prüfer nicht abdeckt.

- **Der neue Prüfer, der fehlt:** zwei **verschiedene** Zeilen mit derselben Kennung in **einem** Lauf, Erwartung: beide erscheinen in der Meldung. Ohne ihn hängt die Korrektur wieder an niemandem. Er gehört in dieselbe Datei.

- **Erhöhte Meldemenge beim ersten Lauf nach dem Fix.**

Steht eine Datei mit vielen ungelesenen Zeilen an, zeigt der Posten fünf davon plus Zählzeile statt einer. Das ist gewollt, kann aber als Lärm empfunden werden; die Obergrenze je Meldung fängt es ab.

- **Was weiterhin still bleibt:** Ein Befund, dessen Kennung **innerhalb der Sperrfrist** erneut auftritt, verschwindet nach wie vor ohne Spur, sobald der Lesestand darüber hinweggeht. Punkt zwei macht ihn wenigstens zählbar. Ganz auflösen ließe er sich nur, wenn der Lesestand erst wandert, wenn eine Zeile tatsächlich gezeigt wurde — das ist ein größerer Eingriff und hier ausdrücklich **nicht** vorgeschlagen.

- **Zum zweiten Befund:** Bekäme der Postfach-Weg die Möglichkeit, Fragen zu registrieren, könnte **jede** darüber zugestellte Nachricht einen Modelllauf auslösen. Am 24.07. liefen aus genau diesem Grund fünf Läufe in sechzehn Sekunden, deren ganzes Ergebnis „Passt.“ und „Gut.“ war — die stille Quittung ist die Antwort darauf und darf nicht versehentlich rückgängig gemacht werden. Die reine Umformulierung der Schlusszeile hat dieses Risiko nicht.

- **Unberührt bleibt** die Zurückhaltung roter Wortlaute: Der Vorschlag ändert nichts an `_melde_zeile()`. Mehr durchgelassene Befunde heißt aber auch mehr Einstufungen — bei einem Ausfall der Ampel gilt weiterhin Rot, was richtig ist, in der Menge aber auffälliger wird.

Offen

- Die fünf Stundenblumen-Meldungen vom 16.08. sind inhaltlich unbekannt. Ob es sich lohnt, sie aus den Auftragsdateien nachzulesen, entscheidet Engywuck — sie liegen vollständig vor.
- Der Bot-Zustand am 16.08. um 01:47 Uhr ist ungeklärt. Das Muster aus Zeitüberschreitungen und nicht initialisiertem HTTP-Client deutet auf einen Neustart; belegt ist das nicht.

Vermutung, nicht Befund.